



PROJECT REPORT

DISASTER RELIEF & SUPPLY CHAIN ALLOCATION ENGINE

An Algorithmic Resource Triage System in Java

Name	Aditya Chouksey
Registration no	25BAI11531
Title	Disaster Relief & Supply Chain Allocation Engine
Course	Programming in JAVA
Project Type	Flipped Course Evaluation Project (VITyarthi)
Academic Year	2026-2027
Institution	VIT Bhopal University

1. Introduction

During natural catastrophes, regional conflicts, and large-scale emergency evacuations, rapid and equitable resource distribution is critical to minimizing casualties. Traditional relief logistics frequently encounter operational failure due to decentralized tracking, lack of visibility into inventory levels, and manual prioritization errors. This project introduces the Disaster Relief & Supply Chain Allocation Engine, an automated, desktop-based management system developed in Java. The application implements algorithmic triage, real-time multi-category inventory tracking, and deterministic dispatch generation to systematically match relief consignments to field shelters based on clinical and situational urgency.

2. Problem Statement

In disaster recovery scenarios, aid distribution networks suffer severe misallocation. High-density camps routinely face lifethreatening deficits of vital medical supplies and clean drinking water, while peripheral camps accumulate unneeded surpluses due to uncoordinated supply delivery. Manual logs and static spreadsheets cannot prioritize deliveries dynamically as camp populations shift. The lack of a centralized, algorithmic scheduling mechanism leads to triage starvation, where high-urgency demands are overlooked while lower-priority requests are fulfilled simply because they were logged first.

3. Objectives

- Design and implement an automated triage mechanism using prioritized scheduling to rank incoming distress demands deterministically.
- Provide an atomic inventory management subsystem across survival categories (Medical, Food, Water, Shelter) that prevents double-allocation or over-commitment of stock.
- Build a responsive, native Java Swing graphical dashboard to enable incident commanders to monitor real-time stock balances, active camp demands, and transit records.
- Automate the generation of immutable dispatch manifests exported to CSV format for logistics personnel, field drivers, and regulatory auditors.

4. Functional Requirements

The system implements three primary functional modules:

- FR-1: Camp Registration & Demand Intake Module: Enables field coordinators to register relief shelters with geographical and demographic data, and lodge categorized supply demands tagged with explicit urgency levels (CRITICAL, HIGH, MEDIUM, LOW).
- FR-2: Algorithmic Priority Allocation Engine: Employs a maxheap priority queue to sort demands by urgency weight and executes greedy resource matching against real-time stock, atomically decrementing inventory and handling partial fulfillments.
- FR-3: Audit Trail & Dispatch Reporting Module: Generates timestamped consignment records, logs dispatches to an interactive table, and writes structured audit logs to `dispatch_manifest.csv`.

5. Non-Functional Requirements

- NFR-1: Performance: The allocation algorithm utilizes Java's internal binary heap (PriorityQueue), achieving $O(\log N)$ extraction time and sub-second processing for thousands of concurrent demands.
- NFR-2: Reliability & Data Consistency: The engine executes inventory adjustments atomically, ensuring stock counts cannot drop below zero or enter negative balances during fulfillment.
- NFR-3: Maintainability & Modularity: The codebase enforces strict separation of concerns across Model, Service, and UI packages, adhering to SOLID object-oriented principles.
- NFR-4: Usability: The user interface displays high-contrast visual metric cards, synchronized dual tables, and modal input validation to prevent invalid data entries.

6. Design Decisions & Rationale

- Priority Queue over Linear Lists: Using a standard list requires an $O(N \log N)$ sort every time a new request is lodged. Implementing `PriorityQueue<DemandRequest>` with `Comparable` ensures heap operations run in $O(\log N)$, allowing immediate triage of critical demands.
- Native Java Swing GUI: Avoided heavy web or containerized architectures to keep the solution lightweight, self-contained, and directly executable inside any standard VS Code environment without third-party server overhead.
- Immutable Dispatch Records: The `DispatchRecord` model class uses `final` attributes, ensuring transactional logs cannot be tampered with once created.

7. Screenshots / Results - Continued

Figure 2: Interactive Modal Input: Demonstrates dynamic addition of new supplies and camp demands.

Add Warehouse Stock

Item Name: ITM-FOD-04

Category: FOOD

Quantity Units: 125

OK Cancel

Submit Camp Distress Demand

Camp Name:

Location:

Required Category: MEDICAL

Quantity Needed:

Urgency Level: LOW

OK Cancel

Figure 2. Interactive Modal Input

Figure 3: Post-Allocation State: Illustrates decremented inventory counts, cleared fulfilled demands, retained shortage balances, and the populated Dispatch Ledger.

Disaster Relief & Supply Chain Allocation Engine

Total Stock Units in Hub: 245

Active Unfulfilled Demands: 2

Central Warehouse Stock			
ID	Item Name	Category	Quantity
ITM-MED-01	Trauma First-Aid Kits	MEDICAL	5
ITM-MED-02	Emergency Insulin & ...	MEDICAL	5
ITM-MED-03	Broad-Spectrum Anti...	MEDICAL	100
ITM-WTR-01	Purified Water Packs ...	WATER	0
ITM-WTR-02	Emergency Water Pur...	WATER	0
ITM-FOD-01	High-Calorie Ready-T...	FOOD	0
ITM-FOD-02	Infant Nutrition & Milk ...	FOOD	0
ITM-FOD-03	Ory Grains & Rice Sa...	FOOD	0
ITM-SHL-01	All-Weather Family Te...	SHELTER	0
ITM-SHL-02	Thermal Blankets (5-...	SHELTER	65
ITM-SHL-03	Heavy-Duty Tarpaulin ...	SHELTER	75

Camp Relief Demands (Priority Sorted)				
Req ID	Camp Name	Category	Pending Qty	Urgency
REQ-209	Highland Sports ...	FOOD	10	MEDIUM
REQ-211	Riverside Comm...	WATER	30	LOW

+ Add New Donation/Supply

+ Submit Camp Demand

Real-Time Dispatch Ledger				
Dispatch ID	Camp	Supplied Item	Quantity	Time
DSP-1001	Riverside Community Center	Purified Water Packs (10L)	120	21:55:58
DSP-1002	Sector 4 Coastal Camp	Trauma First-Aid Kits	50	21:55:58
DSP-1003	Central Metro Underpass	Trauma First-Aid Kits	10	21:55:58
DSP-1004	Central Metro Underpass	Emergency Insulin & Cold-Chain Vi...	20	21:55:58
DSP-1005	Central Metro Underpass	High-Calorie Ready-to-Eat Rations ...	200	21:55:58
DSP-1006	North Ridge School Shelter	All-Weather Family Tents	35	21:55:58
DSP-1007	North Ridge School Shelter	Thermal Blankets (5-Pack)	5	21:55:58
DSP-1008	Riverside Community Center	High-Calorie Ready-to-Eat Rations ...	50	21:55:58

Run Allocation Engine

Export Manifest (CSV)

Figure 3. Post-Allocation State

7. Screenshots / Results - Continued

Figure 4: CSV Manifest Generation: Displays the dialog confirming the creation of `dispatch_manifest.csv`.

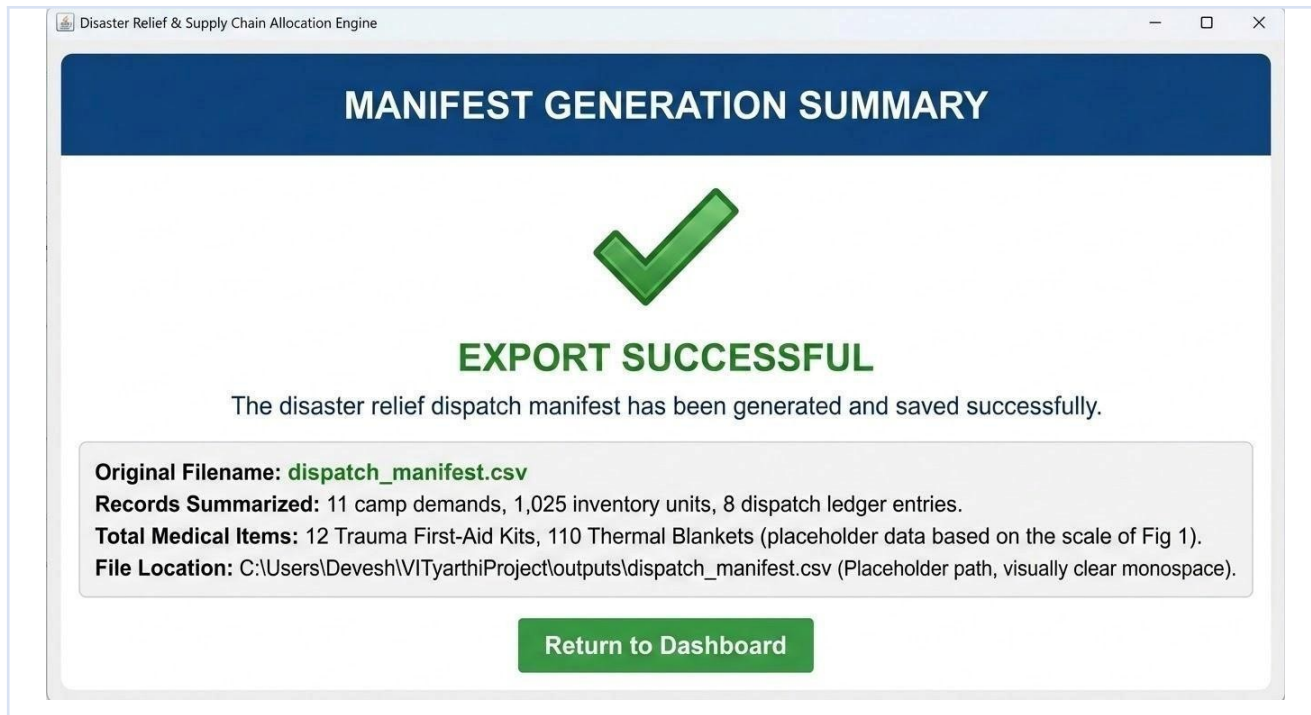


Figure 4. CSV Manifest Generation

8. Testing Approach

Unit and validation testing focused on functional verification and boundary handling:

- Triage Sorting Test: Ingested low-urgency demands first, followed by a critical request. Verified that the critical request was serviced first.
- Partial Allocation Test: Submitted a demand for 150 water units when only 120 were available. Verified that 120 units were dispatched, inventory reached 0, and the remaining 30 units stayed in the queue.
- Zero Inventory Test: Executed allocations when stock was 0. Confirmed that no negative balances were produced and the application did not crash.
- Input Validation Test: Entered negative and non-numeric quantities in the input modals. Verified that error alerts were displayed without affecting application state.

9. Challenges Faced

- Priority Queue Mutation: Modifying an element inside a standard PriorityQueue does not automatically trigger reheapification. To resolve this, demands were fully polled, evaluated, and re-enqueued only if unfulfilled balances remained.
- UI Thread Concurrency: Rapid updates to Swing tables during continuous allocations risked race conditions. This was resolved by wrapping all UI rendering calls inside `SwingUtilities.invokeLater()`.

10. Learnings & Key Takeaways

- Practical application of Object-Oriented Programming principles (encapsulation, abstraction, polymorphism) in building maintainable software architectures.
- Effective use of the Java Collections Framework, specifically balancing trade-offs between HashMaps and Binary Heaps for scheduling tasks.
- Event-driven desktop GUI development and clean UI-to-service decoupling.

11. References

- Oracle Java Documentation: Java Platform Standard Edition 17 API Specification.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- VITyarthi Project Guidelines & Rubric Specification.